Data Dictionary

- A *data dictionary* stores **metadata**, namely data about the data in the database
- There is no universal standard for data dictionary
  - Individual DBMS has the data dictionary in its own way
  - Some DBMS include data governance policies as well as security and privacy policies
- A **passive data dictionary** is used for **documentation** purposes
- An **active data dictionary** is a sophisticated one that interacts with the "mother database" in a real-time fashion
- Generally, the *database schema* (i.e., ER model) is a subset of the data dictionary

An Example of Data Dictionary

"Mother Database"

CLASSROOM Relation			
* ClassroomID	ClassroomLocation	ClassroomType	ClassroomSize

COURSE Relation				
* CourseID	CourseName	CourseEnrollment	CourseTime	*ClassroomID*

Data Dictionary

Relations Metadata

Table Name	Number of Records	Disk Cylinder	Access Permission code
CLASSROOM	232	A234	103
COURSE	568	M467	347

Attributes Metadata

Attribute Name	Data Type	Attribute Length	Indexed	Validation Rule Code
ClassroomID	Text	6	Key / FK	12
ClassroomLocation	Text	10	No	23
ClassroomType	Text	10	No	23
ClassroomSize	Number	3	No	35
CourseID	Text	6	Key	12
CourseName	Text	30	Yes	23
CourseEnrollment	Number	3	No	35
CourseTime	Date/Time	16	No	11

Database Transactions

- A database transaction is a single, atomic "*unit of work*" that groups multiple database operations into one logical whole, ensuring that all operations succeed, or none of them do, in order to maintain data integrity
  - Bundling several database queries, such as inserts, updates, or deletes, into a single unit, which generally corresponds to a business transaction (e.g., bank transfer)
  - All-or-nothing approach: essential for preventing data corruption and achieving data integrity, concurrency control and fault tolerance
    - **Commit** – if all operations within the transaction are successful, the changes are permanently saved to the database
    - **Rollback** (or **Abort**) – if any operation in the transaction fails, all the previous steps are undone, and the database is returned to its original state, as if the transaction never happened

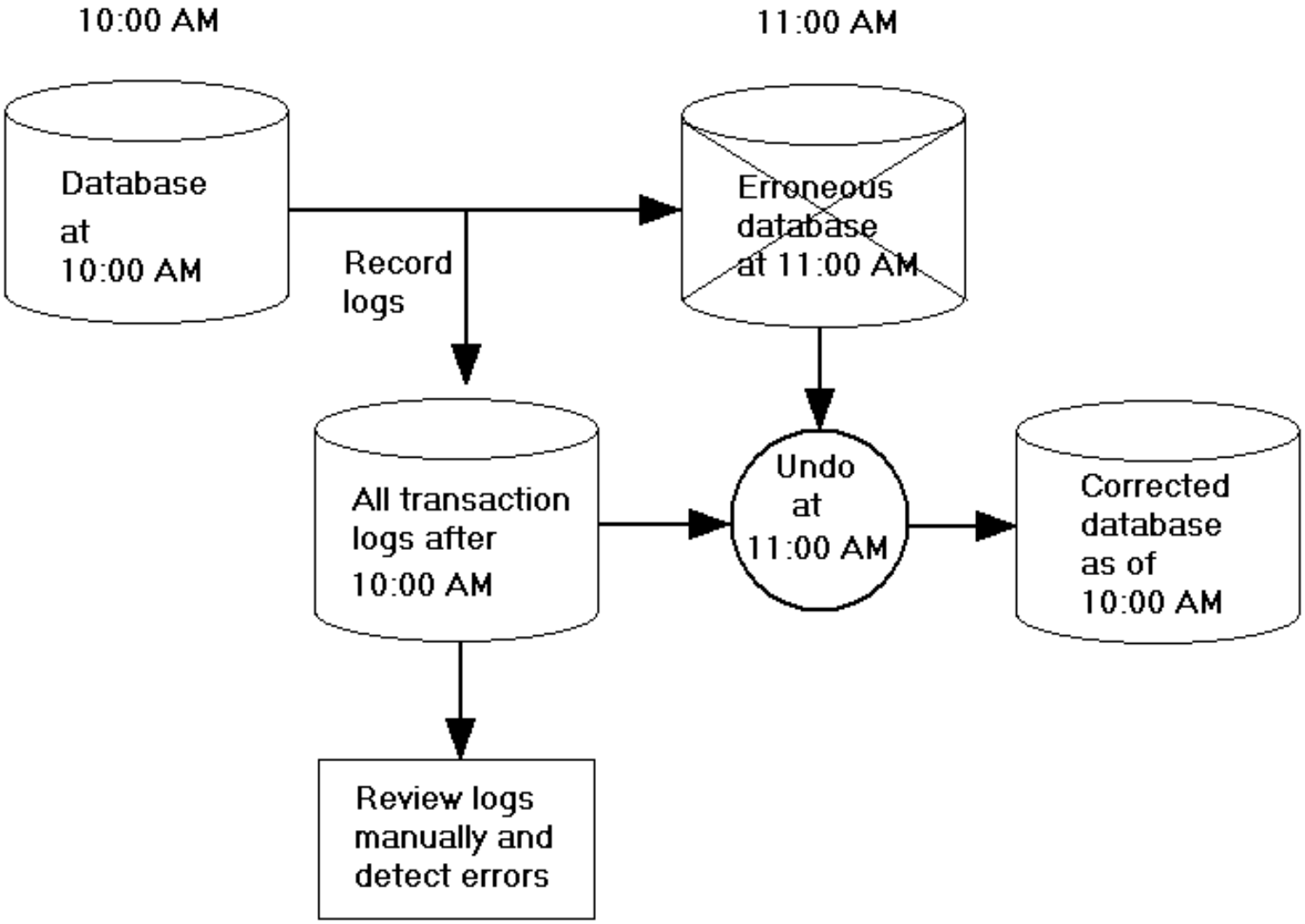
Database Transactions

- Data Integrity
  - Transactions prevent partial updates, ensuring that the database remains in a consistent and reliable state
- Concurrency Control
  - Transactions manage how multiple users and processes access the database simultaneously, avoiding conflicts and data anomalies
- Fault Tolerance
  - Transactions allow for recovery from failures by rolling back incomplete operations, maintaining the database's integrity even when errors occur

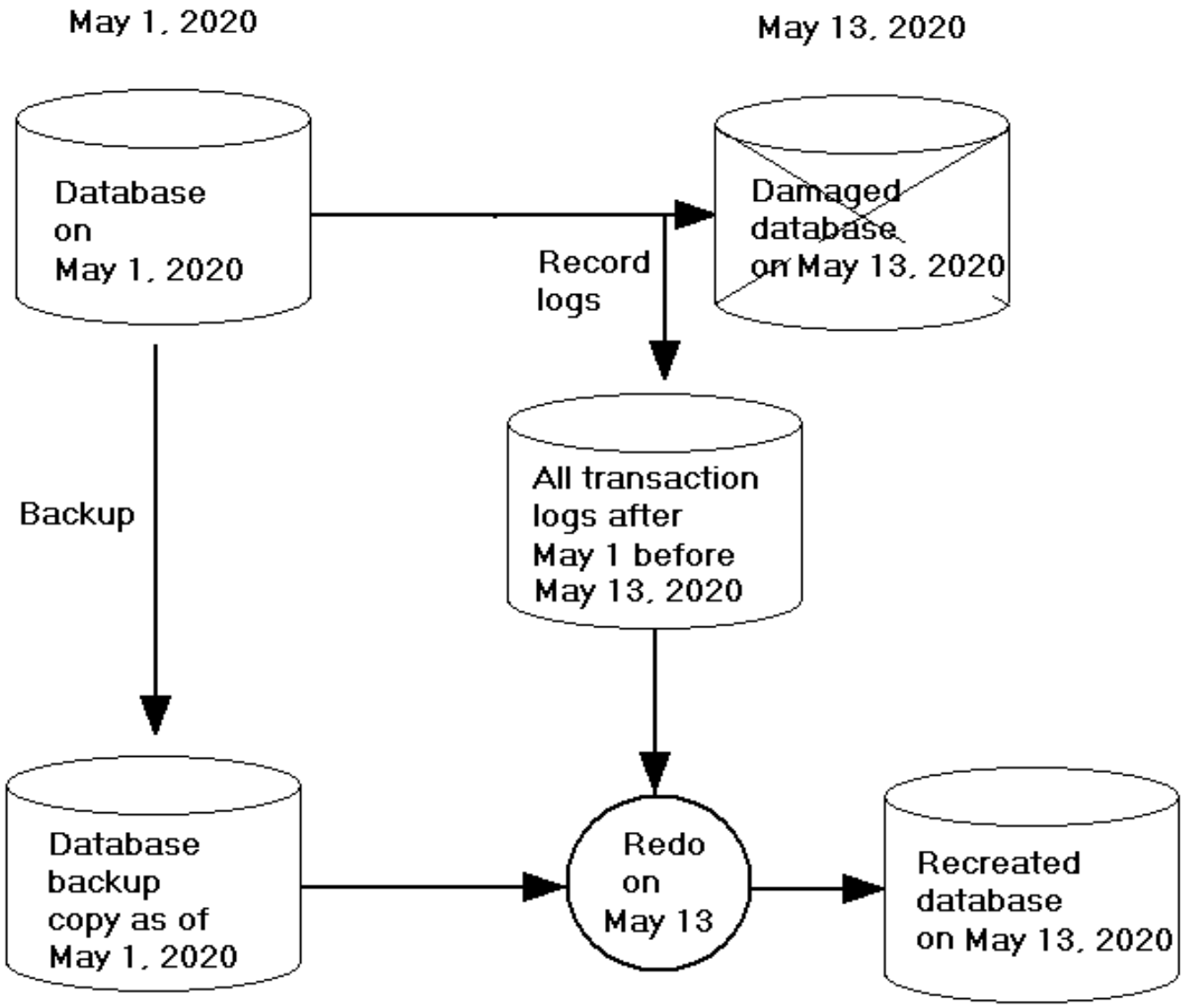
ACID Properties of Database Transactions

- Atomicity
  - Ensure all operations in a transaction are treated as a single, indivisible unit; either all succeed or none do
- Consistency
  - Guarantee a transaction will bring the database from one valid state to another, maintaining data integrity and adhering to defined rules
- Isolation
  - Ensure concurrent transactions do not interfere with each other; the execution of one transaction does not affect the execution of another
- Durability
  - Guarantee once a transaction is committed, its changes are permanent and will survive system failures like power outages

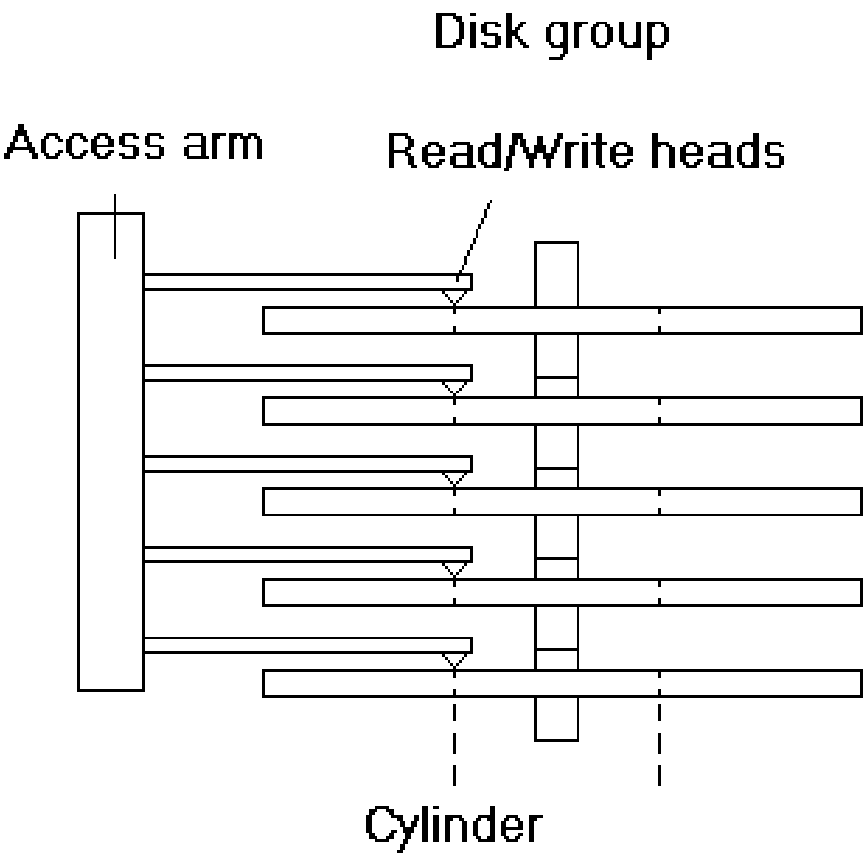
Un-Do Transactions

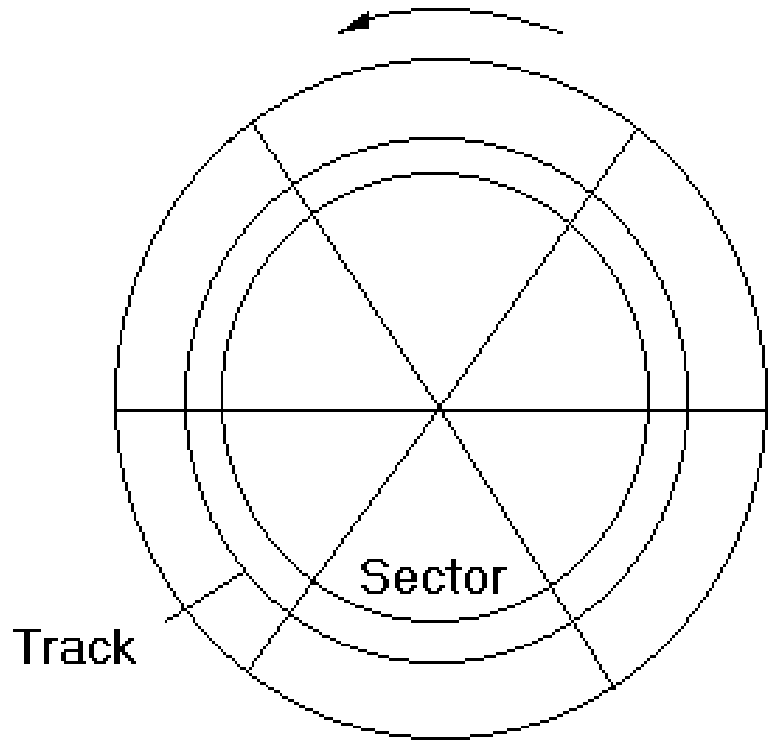


Re-Do Transactions

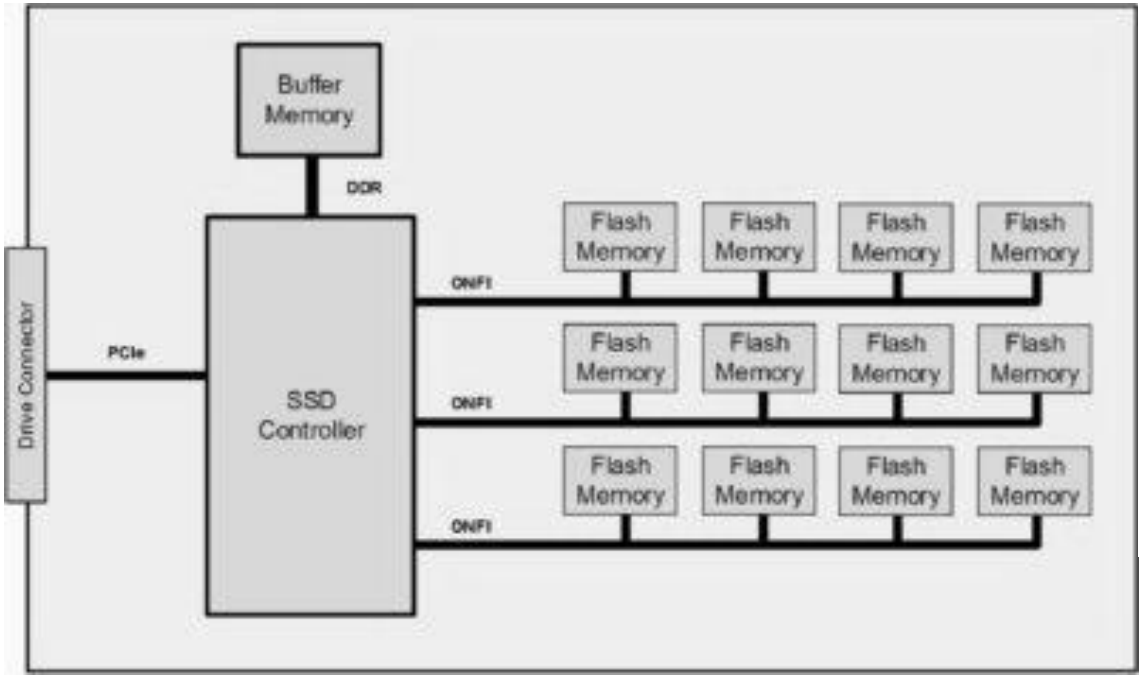


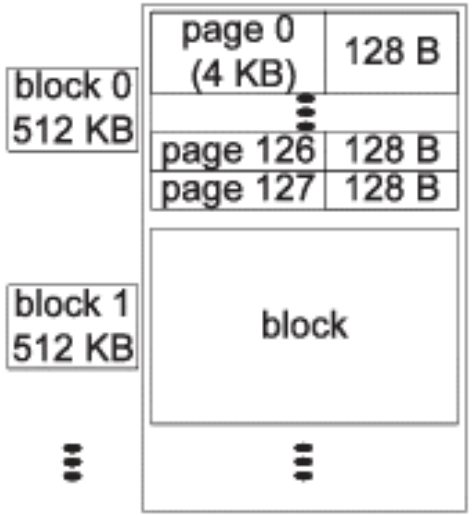
Data Storage – Mechanical Hard Disk





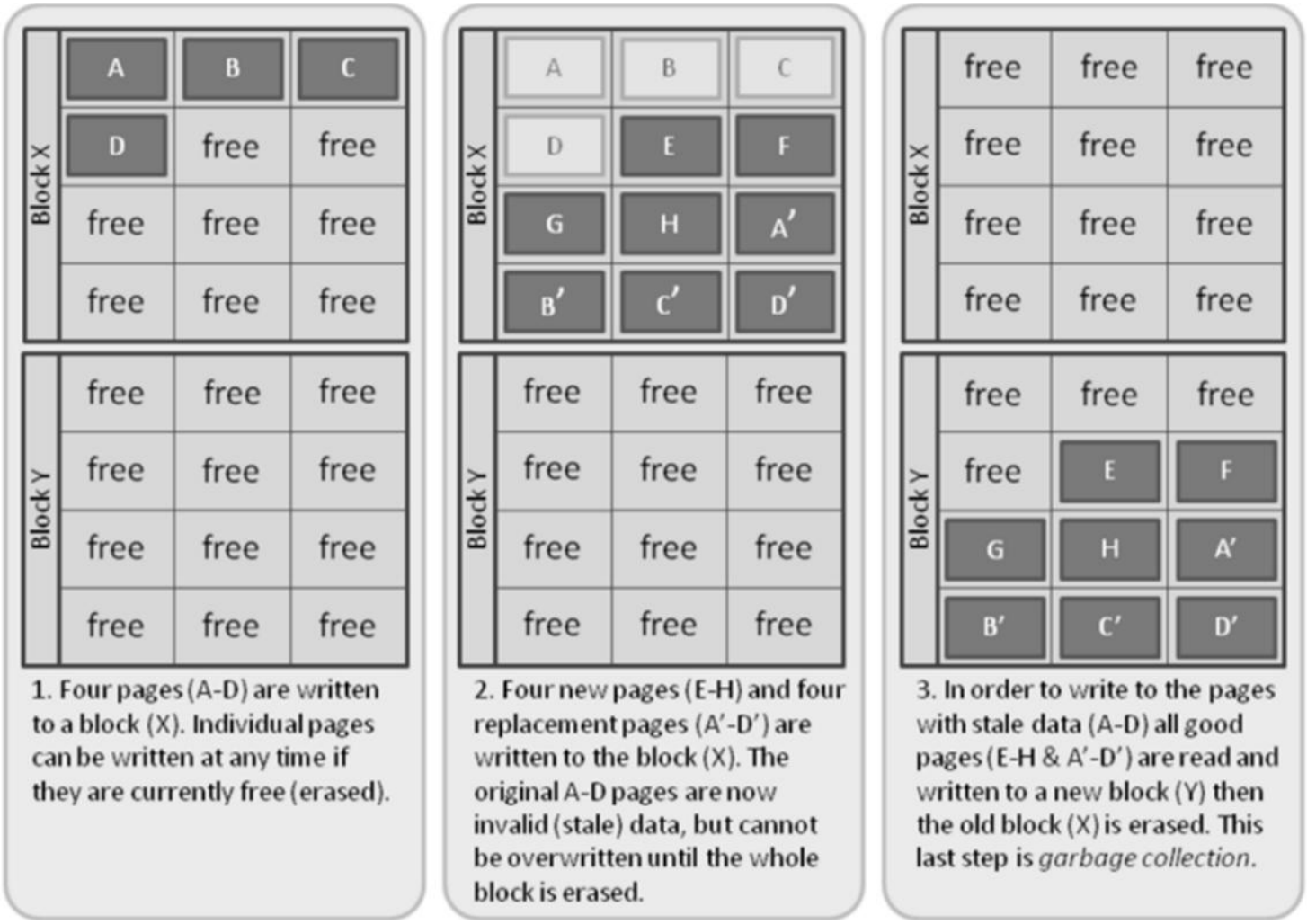
Data Storage – Solid State Drive





NAND Flash Memory

Data Storage – Solid State Drive

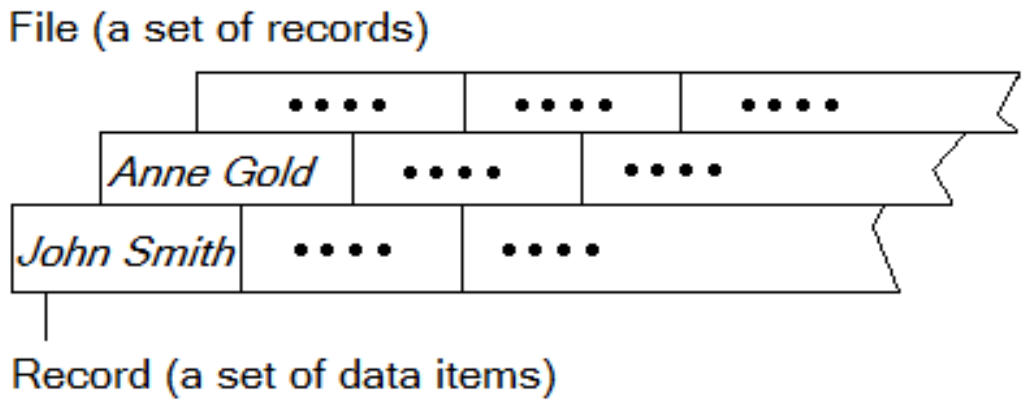


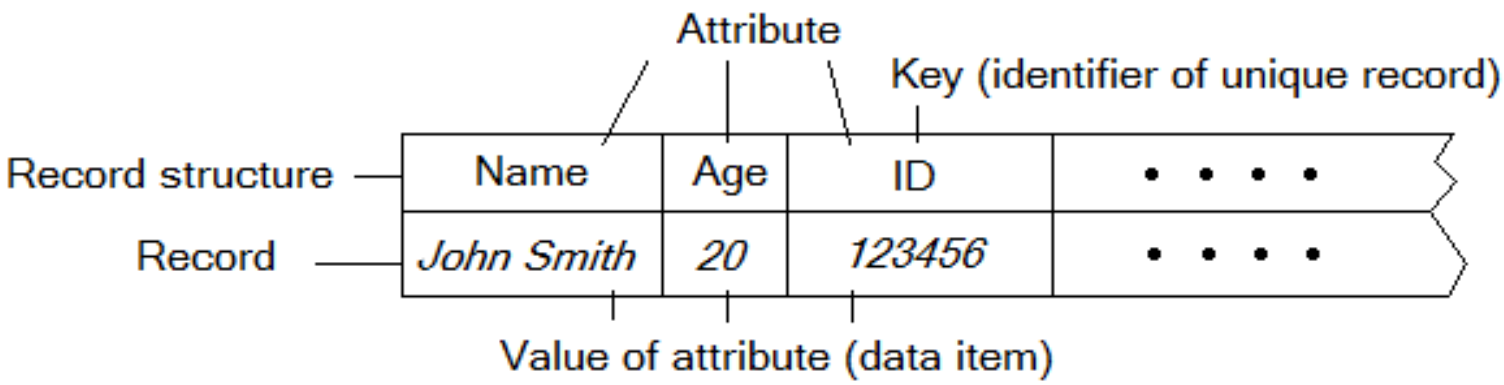
1. Four pages (A-D) are written to a block (X). Individual pages can be written at any time if they are currently free (erased).

2. Four new pages (E-H) and four replacement pages (A′-D′) are written to the block (X). The original A-D pages are now invalid (stale) data, but cannot be overwritten until the whole block is erased.

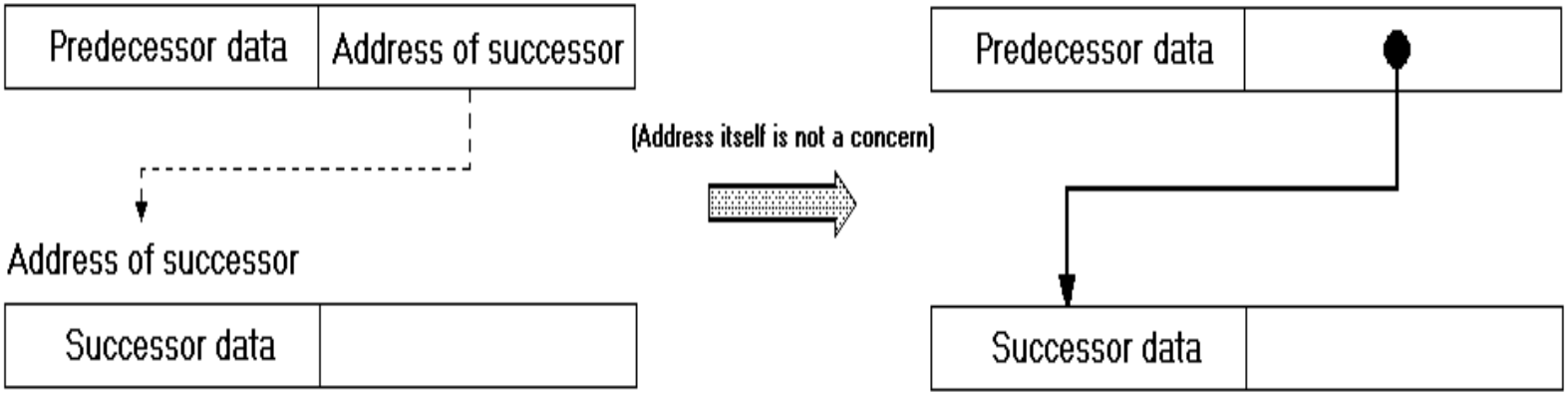
3. In order to write to the pages with stale data (A-D) all good pages (E-H & A′-D′) are read and written to a new block (Y) then the old block (X) is erased. This last step is *garbage collection*.

File, Record, Attribute, and Key





Pointer – a basic technique for data search



Basic File Organizations

- File organizations on disk is critical for data access efficiency
- There are four basic file organizations:

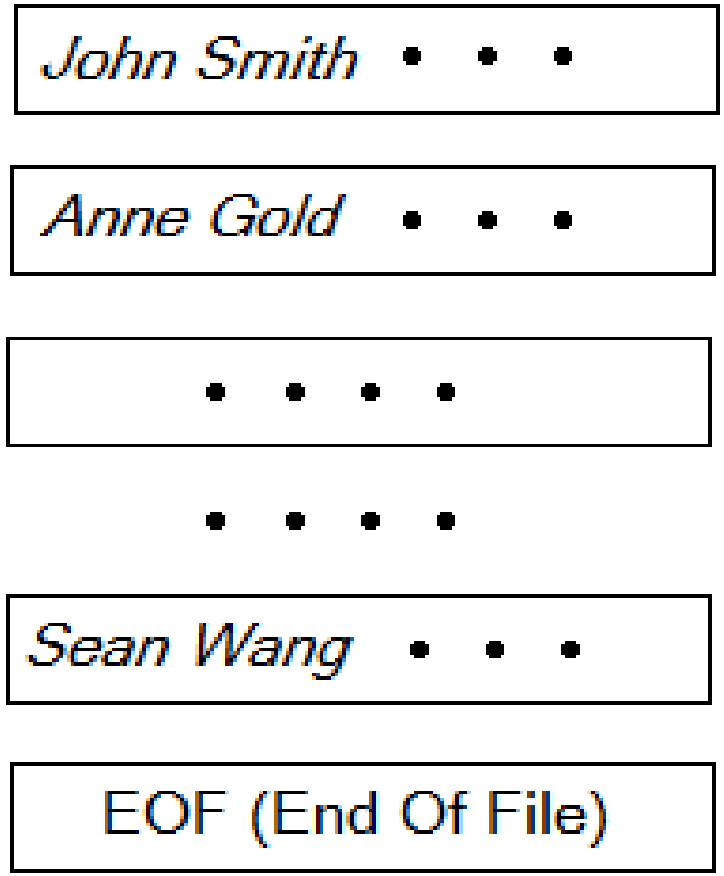
  Sequential

  Random

  Linear indexed

  B^+-tree

Sequential File

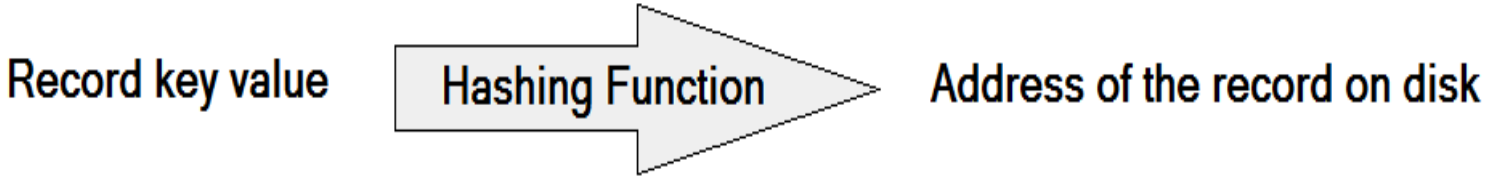


Pros/Cons of Sequential Files

- <u>Advantages of sequential files</u>:
- (1) Saves space
- (2) No record key is required
- (3) Efficient when all of the records are sequentially processed (e.g. payroll processing)
-
- <u>Disadvantages of sequential files</u>:
- (1) It would take a long time to find a particular record
- (2) It is difficult to update. For example, it is difficult to insert a record

Random Files

- Hashing function



- Random files are less used in modern databases

Pros/Cons of Random Files

- <u>Advantages of random files</u>:
- (1) It can access an individual record very fast
- (2) It is efficient in updating (e.g., adding and modifying records)

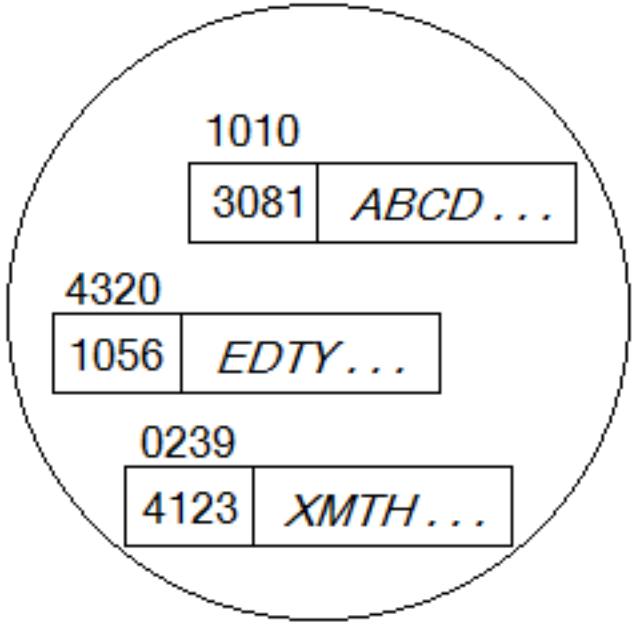
- <u>Disadvantages of random files</u>:
- (1) Sequential access is impossible
- (2) A record key is necessary
- (3) Wastes spaces
- (4) Synonyms make the process slower

(Linear) Indexed File

Index Table

Record Key	Address of Record
3081	1010
4123	0239
1056	4320
• • • •	• • • •

Disk



Pros/Cons of Linear Indexed Files

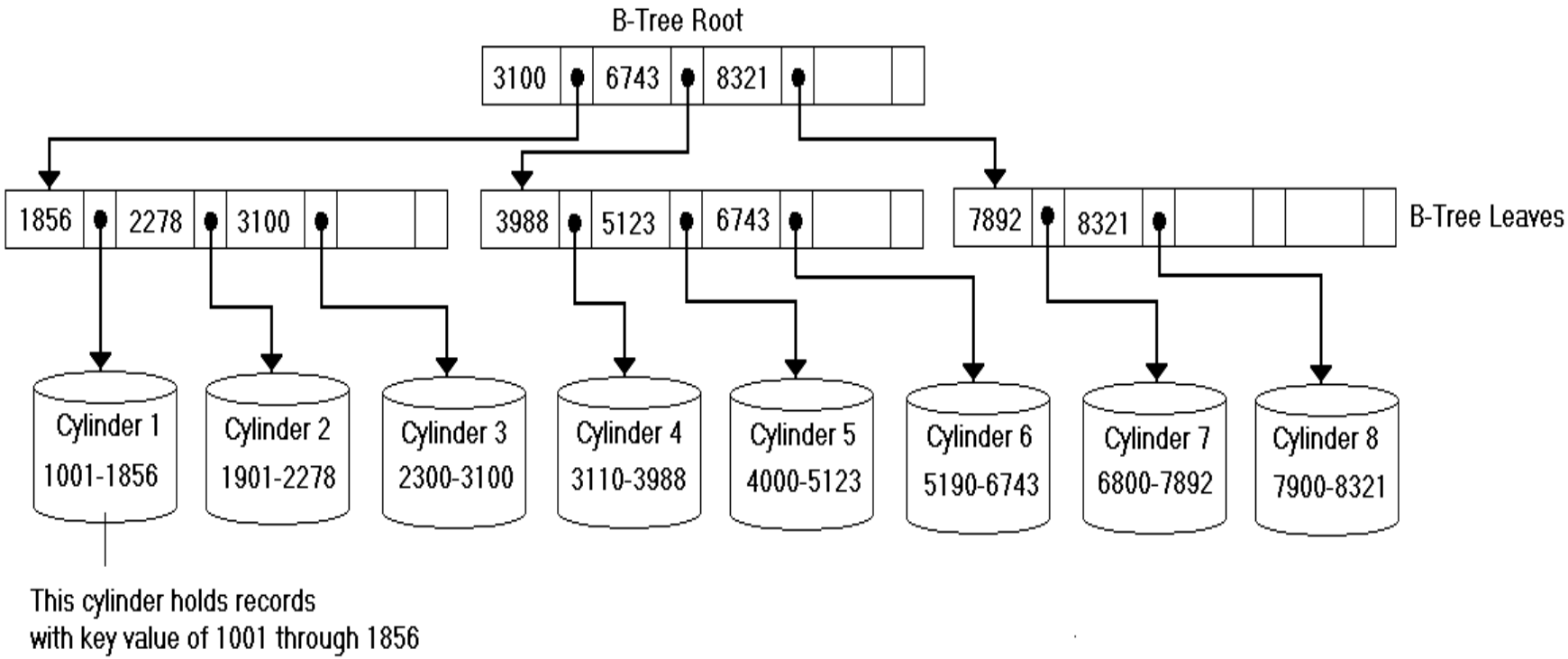
- <u>Advantages of indexed files</u>:
- (1) It can be used in both sequential and random access. Processing speed is fairly good for both
- (2) It is efficient in adding a record, sorting and updating the file, if the indexed file is not huge

- <u>Disadvantages of indexed files</u>:
- (1) A record key is necessary
- (2) If the data file is huge, then the linear index table becomes a sequential disk file, and processing will be slow

B^+-Tree

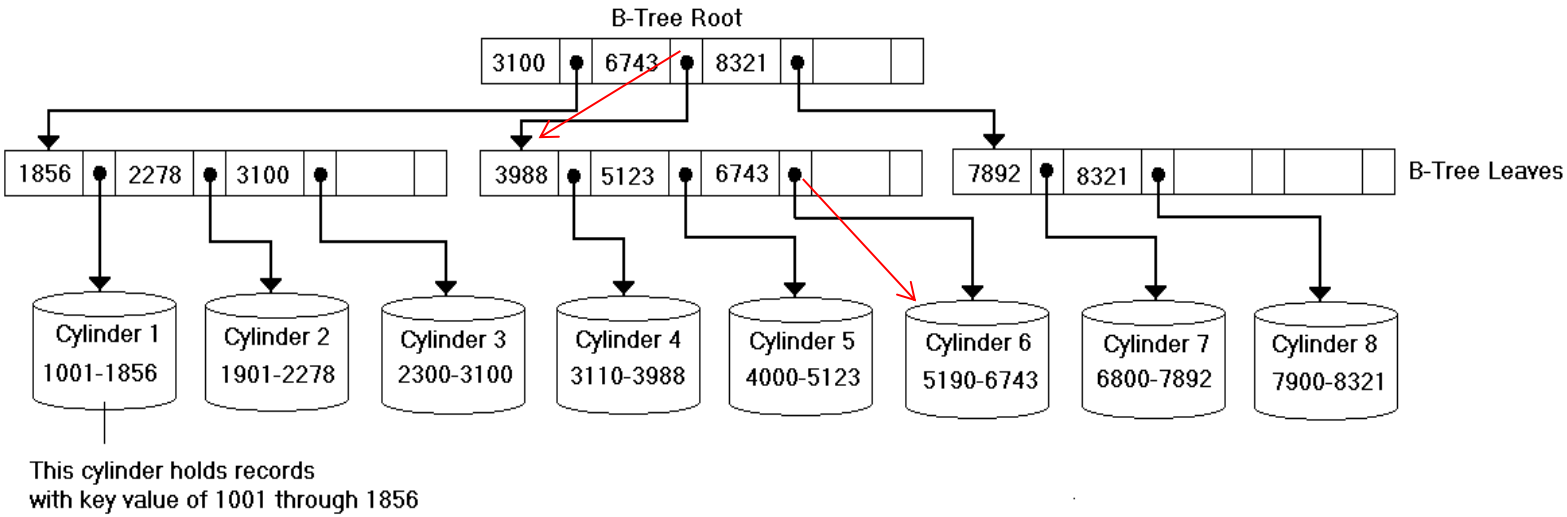
- B^+-tree is the major file organization used in modern databases
- B^+-tree makes databases so powerful in data search
  - All modern DBMS employs B^+-tree as the default indexing method

B^+-Tree


B-Tree Root
3100
6743
8321
1856
2278
3100
3988
5123
6743
7892
8321
B-Tree Leaves
Cylinder 1
1001-1856
Cylinder 2
1901-2278
Cylinder 3
2300-3100
Cylinder 4
3110-3988
Cylinder 5
4000-5123
Cylinder 6
5190-6743
Cylinder 7
6800-7892
Cylinder 8
7900-8321
This cylinder holds records
with key value of 1001 through 1856

B^+-Tree:

Find record with key value of 6000



Rules in B^+-Tree

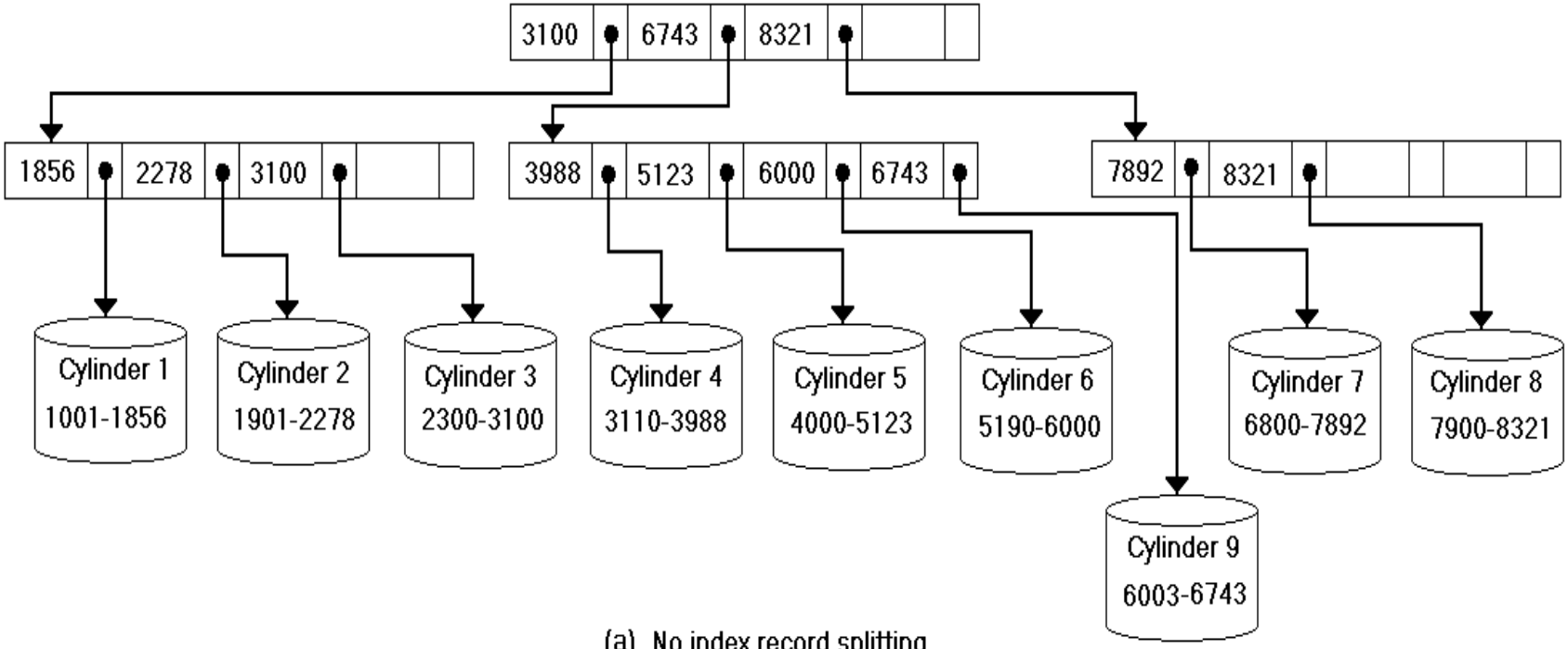
- (1) A B^+-tree consists of index records that are organized into a tree. The tree has its root with one and just one index record on the top, and a number of levels
- (2) Each index record accommodates the same number of pairs (4 in the example in previous slides), and each pair can hold a key-value and a pointer
- (3) Each index record is at least half full (in the example in previous slides, at least 2 pairs in an index record are occupied with key-values and pointers, and the rest pairs can be empty)
- (4) In each occupied key-value-pointer pair, the pointer points to an index record at the next level or a disk cylinder if the pair is contained in a leaf index record, and the key-value is the maximum key value in the pointed index record or the maximum key value on the cylinder

B^+-tree construction – from bottom to up

- Step 1. List all given disk cylinders in the ascending order of their maximum key values of records
- Step 2. Given a certain number of key-value/pointer pairs of each index record, determine the number of index records at the leaf level for the given cylinders, and create the index records
- Step 3. Following the construction rules, fill each index record, and link the pointers to the cylinders
- Step 4. Determine the number of index records for the next necessary higher level, and create the index record(s)
- Step 5. Following the construction rules, fill each index record, and link the pointers to the lower level index records
- Step 6. Repeat Step 4 through Step 5 until the root of the tree has been reached

B$^+$-Tree Maintenance

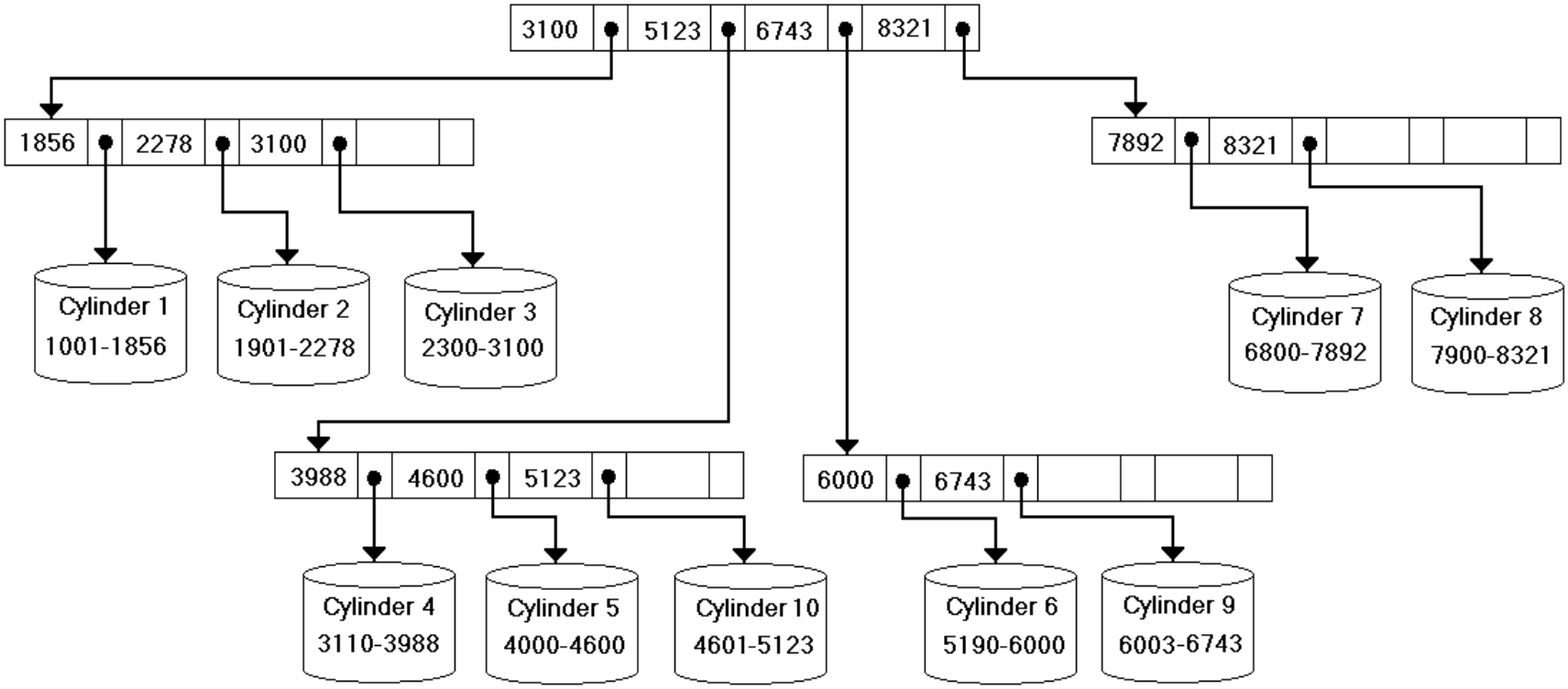
Example: *Cylinder 6 is full and add Cylinder 9*



(a) No index record splitting

B$^+$-Tree: Maintenance

Example: *Cylinder 5 is full and Cylinder 10 is added: Splitting index records Splitting index records*



(b) Index record splitting and upper-level index record updating

Summary of B^+-Tree

- In our examples, B^+-tree is used for indexing key values. Actually, B^+-tree can also be used for non-keys
- A database management system can construct and maintain a huge amount of B^+-trees so that random data access would be efficient
- On the other hand, significant computational resource is devoted to B^+-tree construction and maintenance

Physical Design

- **Physical database design** is to alter the logical database design and improve the database performance

- **Response time** – the time delay from the database access request to the result appears
- T**hroughput volume** – the data access output in a given time period
- S**ecurity** – the data protection

Physical Database Design Techniques

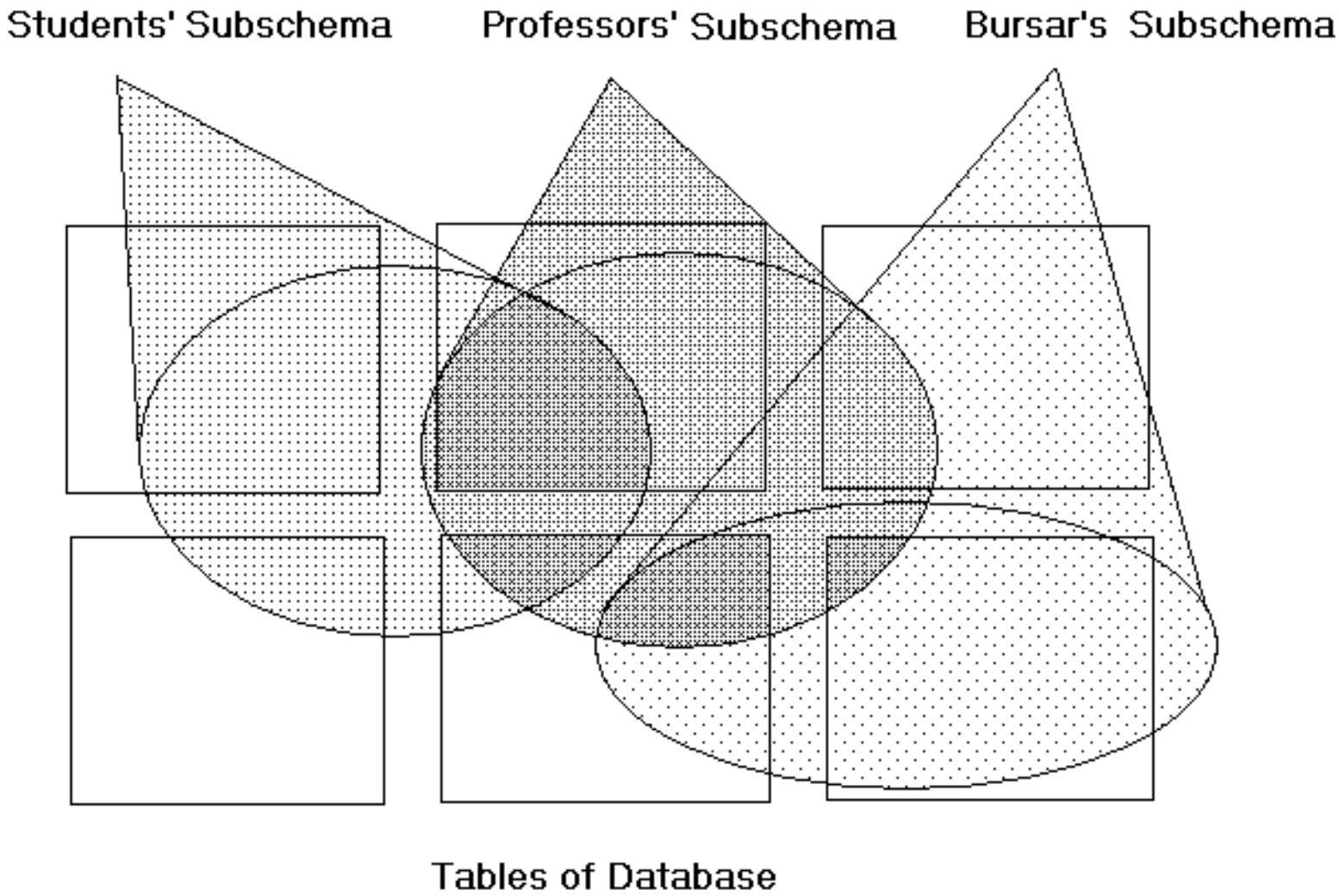
- Unlike logical database design, physical database design is more or less artistic and depends upon the specific situations of applications of the database
- The support functions for physical design vary depending on the DBMS. Large DBMS such as Oracle and DB2 have rich functionalities for physical database design, but end user oriented DBMS for tiny databases, such as Microsoft Access, have few physical design features

Adding Index

- For example, if the user finds students from the STUDENT table often by StudentName, then StudentName should have an index although it is not the primary key

- The downside of adding indexes is the increase of computation overhead for maintenance of B-trees

Adding Subschema



Clustering Tables

- If two or more tables are used together so often, these tables should be **clustered**; that is, are placed close each to other (e.g., on the same disk cylinder)

- The downside of clustering is that it might increase the “distances” between one of the clustered table and other tables because not all tables can be clustered

Merging Tables

- If two or more tables are repeatedly joined through the join operations, then **merging** these tables into a large table might be necessary to reduce the valuable computing time spent on the repeating join operations
- The **combined table** becomes a **denormalized** table and usually contains data redundancy
- To keep the data sufficiently accurate, re-merging could take place from time to time

Horizontal Partitioning Table

- In **horizontal partitioning**, the rows of a table are divided into parts, and the parts are stored separately for many reasons, such as different access frequencies of the parts and different security requirement of the parts
- For instance, if the STUDENT table is huge, it can be divided into two parts: one sub-table contains records of active students and the other sub-table contains records of inactive students
- The downside of partitioning is that the computation overhead would be increase when the entire table is used

Horizontal partitioning

STUDENT Table

* StudentID	StudentName	StudentAddress	StudentYear
0101	Mark Sousa	12 Northport Ave.	1964
0211	Susan Bush	43 Wood St.	1967
⋮	⋮	⋮	⋮
1201	Chris Smith	23 William St.	2016
1334	Anne Riley	38 Union Ave.	2017
1432	John Kerry	17 Wood Lane	2016
2874	Doug Dale	6 Lake Rd.	2018
⋮	⋮	⋮	⋮

ACTIVE-STUDENT Table

* StudentID	StudentName	StudentAddress	StudentYear
1201	Chris Smith	23 William St.	2016
1334	Anne Riley	38 Union Ave.	2017
1432	John Kerry	17 Wood Lane	2016
2874	Doug Dale	6 Lake Rd.	2018

Vertical Partitioning Table

- In **vertical partitioning**, the columns of a table are divided into parts, and the parts are stored separately because different users require different attributes
- For instance, if the STUDENT table is huge, it can be divided into two parts: one sub-table contains attributes StudentID, StudentName, and StudentYear; and the other sub-table contains attributes StudentID, StudentName, and StudentAddress
- The concepts of vertical partitioning and adding indexes are different

Vertical Partitioning

STUDENT Table

* StudentID	StudentName	StudentAddress	StudentYear
0101	Mark Sousa	12 Northport Ave.	1964
0211	Susan Bush	43 Wood St.	1967
⋮	⋮	⋮	⋮
1201	Chris Smith	23 William St.	2016
1334	Anne Riley	38 Union Ave.	2017
1432	John Kerry	17 Wood Lane	2016
2874	Doug Dale	6 Lake Rd.	2018
⋮	⋮	⋮	⋮

STUDENT-RESIDENCE Table

* StudentID	StudentName	StudentAddress
0101	Mark Sousa	12 Northport Ave.
0211	Susan Bush	43 Wood St.
⋮	⋮	⋮
1201	Chris Smith	23 William St.
1334	Anne Riley	38 Union Ave.
1432	John Kerry	17 Wood Lane
2874	Doug Dale	6 Lake Rd.
⋮	⋮	⋮

Creating New Primary Key

- Combination primary keys make data retrieval process slower, especially when the combination key involves too many attributes, because it makes too many indexed attributes in the B-trees
- For example, [*StudentID + *CourseID + *ProfessorID] is a combination key of a table. One may create a new primary key to replace it. Note that the table will still keep the attributes of the original combination key

Substituting Foreign Key

- If an alternative key of the table on the 1-side is more often used for data retrieval than the primary key, this alternative key could be used as the foreign key of the table on the M-side
- For instance, the PROFESSOR table and TEACH table are in a 1:M relationship with ProfessorID being the primary key in the PROFESSOR table and the foreign key in the TEACH table. Considering ProfessorName is an alternative key of the PROFESSOR table, and is used more frequently than ProfessorID, *ProfessorName* can substitute *ProfessorID* as the foreign key in the TEACH table

Duplicating Table or Parts of Partitioned Table

- When multiple users try to access the same table, the response time of the database will slow. To reduce the bottleneck effect, the technique of **duplicating tables** (or parts of partitioned tables) is used at the risk of overt data redundancy
- The duplication should be temporary for the access peak hours, and should be fully managed by the DBMS

Storing Information (Processed Data)

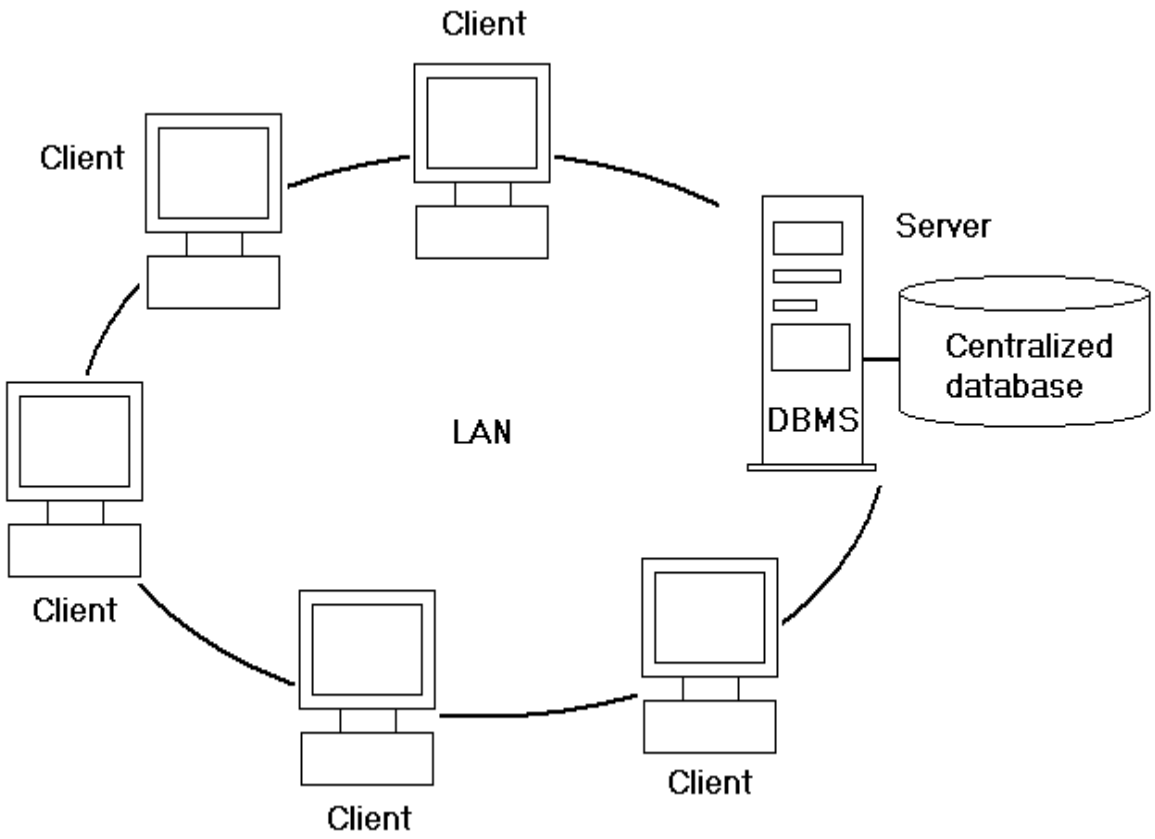
- If a piece of information (or processed data, e.g., total) is requested frequently and/or by many users in a short period, the system would waste much computation resource
- **Storing** information is to reduce the unnecessarily repeating computation
- Apparently, the stored piece of information should be of short life to keep the information updated

Implementation of Physical Database Design

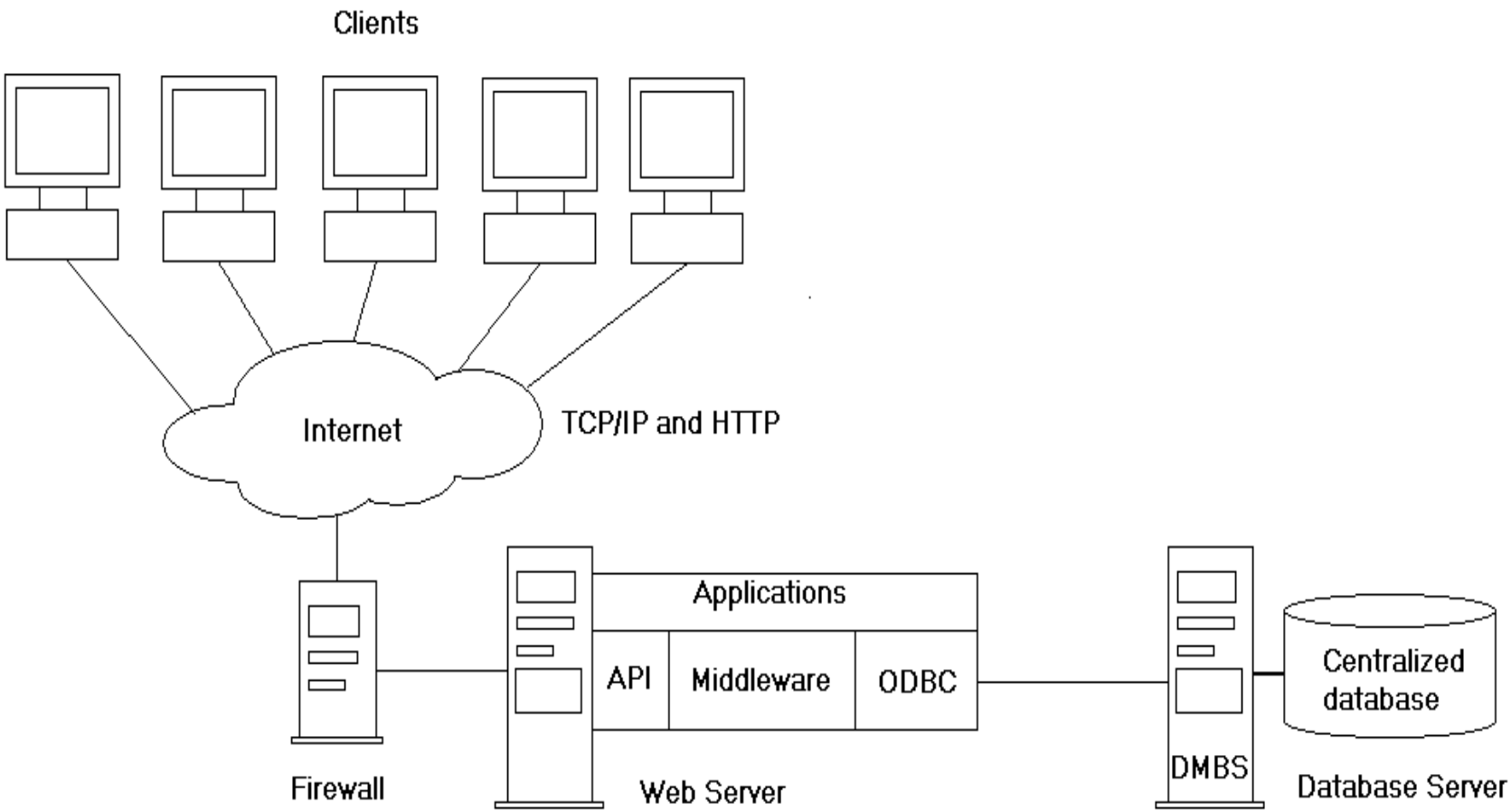
- The implementation of physical database design highly depends on the DBMS

- (1) **Storage-related construction**
- (2) **SQL-related construction**. SQL allows the user of the database to create VIEW. This makes it possible to create a VIEW of multiple tables for security, or to create partitions of a single table
- (3) **Query optimizer**. Many DBMSs support SQL ANALYZE command to analyze database performance statistics. A DBMS has its own algorithms for determine the most efficient way to execute SQL

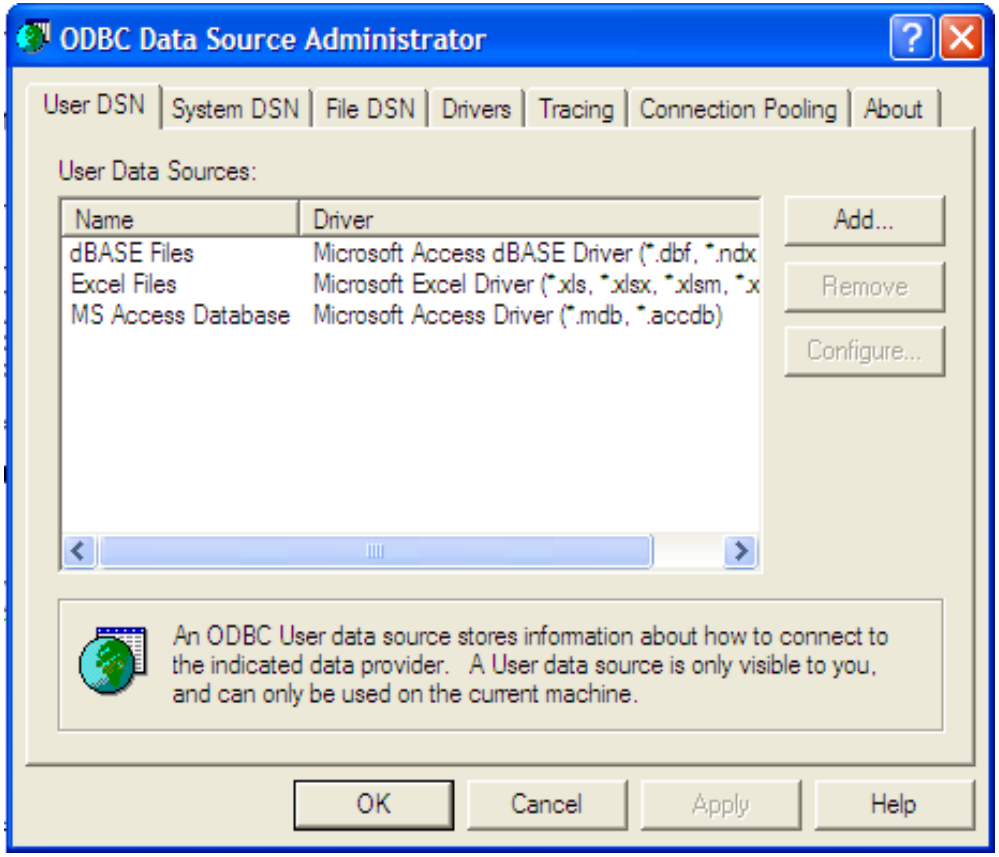
Centralized Database in the Local Area Network Environment



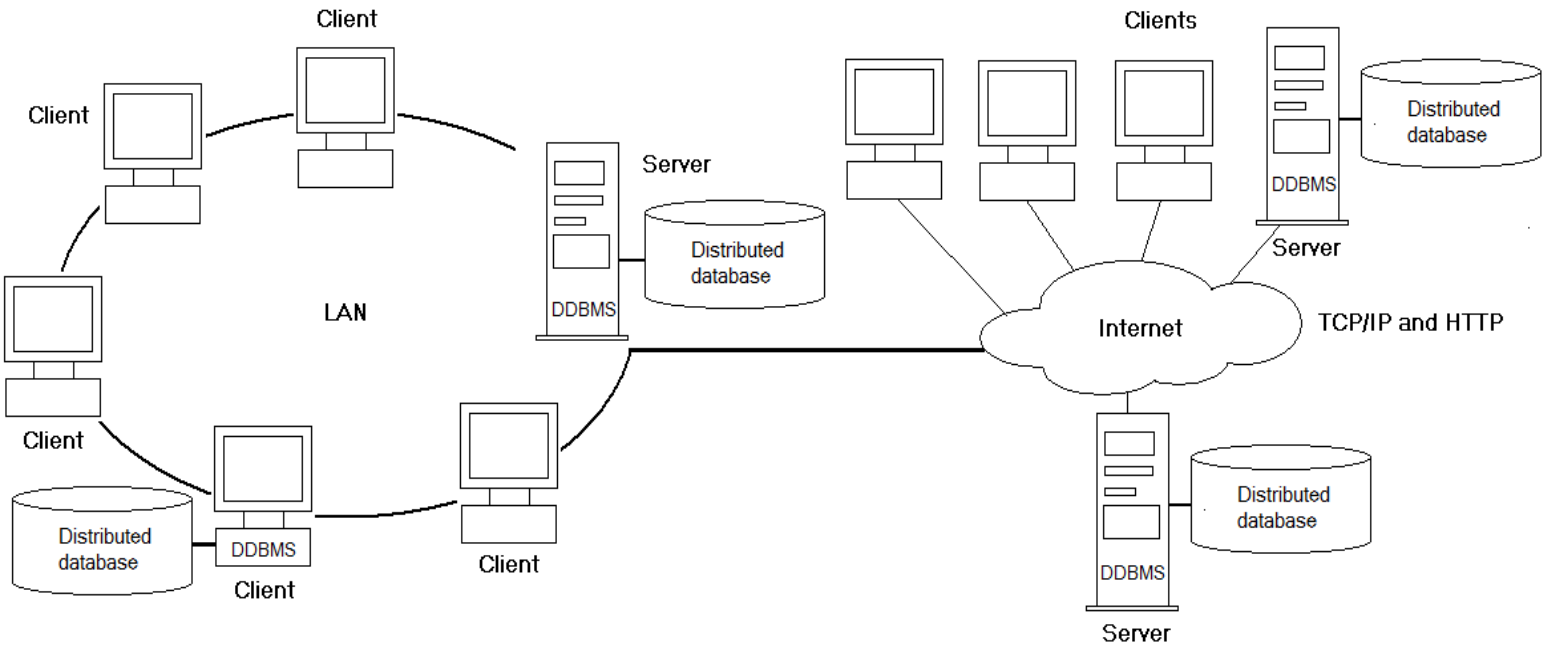
Centralized Database in the Internet Environment



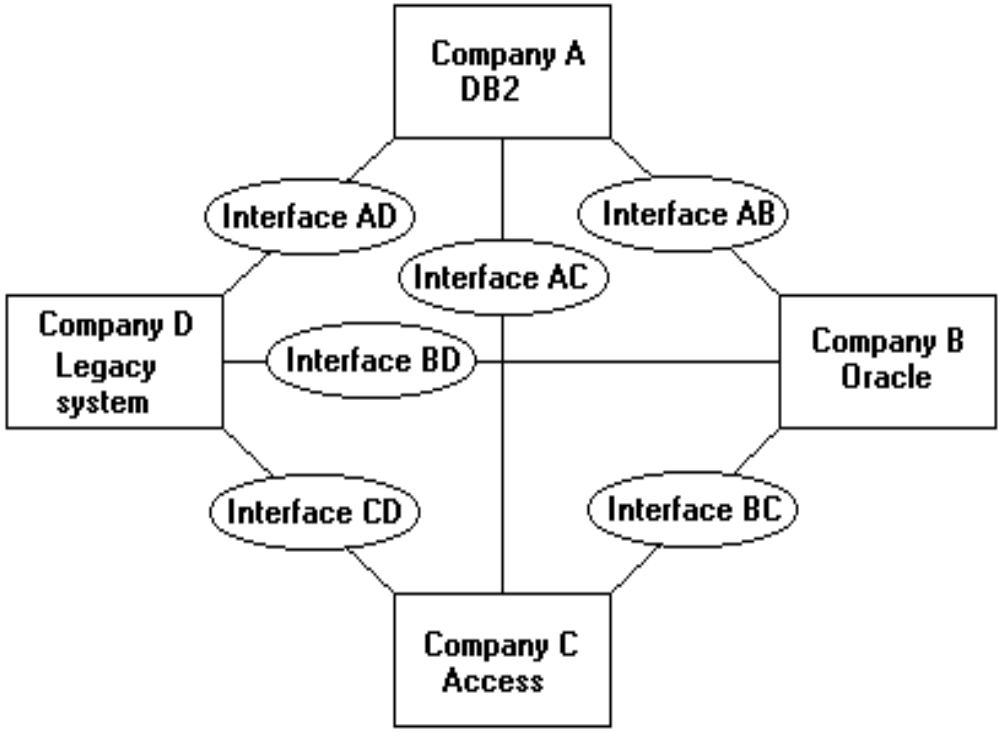
ODBC (Open Database Connectivity)



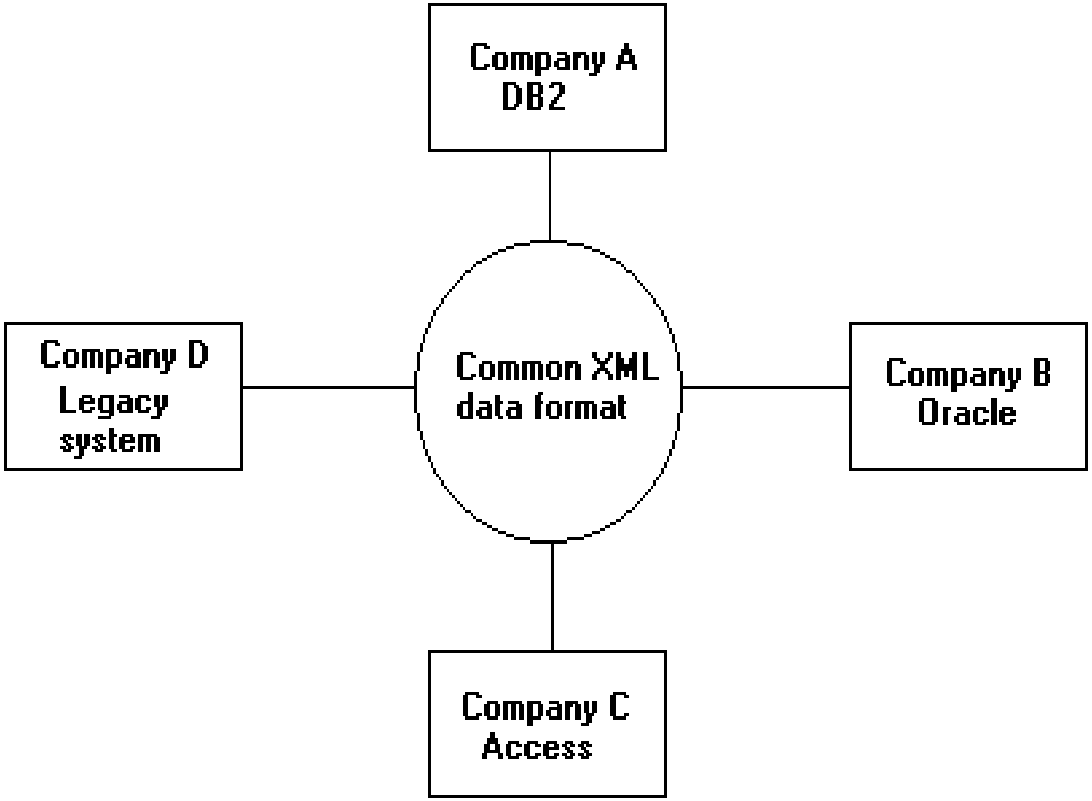
Distributed Database



Interfaces of Databases



XML for databases



XML

- The XML data model is hierarchical, not relational. The conversion of data from a relational database into the XML format might not be straightforward
- The use of XML involves several companion tools, such as XSLT or CSS, XML Schema or DDT, and is not as simple as it should be

An Example of XML for the STUDENT Table

```
<?xml version="1.0" standalone="yes"?>
<StudentTable>
 <Student
          StudentID="01234567"
          StudentName="John"
          StudentAddress="285 Westport"
          StudentEnrolYear="2018" >
 </Student>
 <Student
          StudentID="02345678"
          StudentName="Anne"
          StudentAddress="287 Eastport"
          StudentEnrolYear="2020" >
 </Student>
 <Student
          StudentID="03456789"
          StudentName="Robert"
          StudentAddress="324 Northport"
          StudentEnrolYear="2019" >
 </Student>
</StudentTable>
```